13

Zugriffsrechte und Benutzer verwalten

13.1 Sicherheitskonzepte

Benutzerverwaltung in Datenbanksystemen

Datenbanksysteme speichern häufig sensible Daten, zu deren Schutz ein Sicherheitskonzept mit einer Rechteverwaltung für die Objekte der Datenbank notwendig ist. Dieses Konzept ist bei allen Datenbanksystemen im Wesentlichen ähnlich aufgebaut und unterscheidet sich nur in Details der Umsetzung.

Der wichtigste Schritt im Aufbau eines sicheren Datenbanksystems besteht im Verwalten unterschiedlicher Benutzer bzw. Benutzergruppen. Diesen Benutzern können mit den SQL-Anweisungen `GRANT` und `REVOKE` definierte Zugriffsberechtigungen zugeteilt oder entzogen werden.

Die Benutzerverwaltung übernimmt in der Regel der Datenbank- oder Systemadministrator. Er verfügt über die meisten Rechte im System und kann diese anderen Benutzern zuweisen oder entziehen. Für den Datenbankadministrator wird beim Installieren des Datenbanksystems automatisch ein Benutzerkonto erstellt. Dieser Benutzer hat einen vom verwendeten Datenbanksystem abhängigen Namen.

Datenbanksystem	Benutzername des Administrators	Standard-Passwort
MariaDB	root	keins
PostgreSQL	postgres	keins
Microsoft SQL-Server	sa	keins

! Nach der Installation des Datenbanksystems sollten Sie unbedingt das Passwort des standardmäßig installierten Datenbankadministrators ändern bzw. ein Passwort vergeben, um die Sicherheit des Systems zu bewahren. Da Sie nur für bestimmte Befehle über die Administratorrechte verfügen müssen, sollten Sie (auch für sich selbst) zusätzlich einen Benutzer mit etwas eingeschränkten Rechten erstellen, den Sie als Standardbenutzer einrichten.

© HERDT-Verlag
Schulversion

Bei der Eingabe des Passwortes gelten folgende Regeln:

- Das Passwort kann maximal 32 Zeichen lang sein. Es sind jedoch nur die ersten acht Zeichen relevant.
- Bei der Überprüfung des Passwortes wird, im Gegensatz zum Benutzernamen, zwischen Groß- und Kleinschreibung unterschieden.

Benutzer eines Datenbanksystems

Die Benutzer in einem Datenbanksystem sind unabhängig von den Benutzern des Betriebssystems, auf dem das Datenbanksystem installiert ist. Sie müssen für das Datenbanksystem extra angelegt werden. Die Benutzer des Datenbanksystems sind in der Hierarchie über den einzelnen Datenbanken angeordnet. Daher müssen die Benutzer auch nur einmal angelegt werden. Anschließend kann ein Benutzer dann Zugriff auf alle oder einzelne ausgewählte Datenbanken des Systems erhalten.

Grundsätzlich ist jeder Benutzer, der ein Datenbankobjekt wie z. B. eine Tabelle erstellt, automatisch auch Besitzer dieses Objektes. Alle anderen Benutzer des Systems besitzen für ein Datenbankobjekt eines anderen Benutzers nach dessen Erstellung keine Zugriffsrechte. Eine Ausnahme ist der Datenbankadministrator, der über alle Datenbankobjekte verfügen kann. Er kann auch anderen Benutzern zusätzliche Rechte einräumen.

13.2 Benutzerverwaltung unter PostgreSQL

Mit Rollen arbeiten in PostgreSQL

Sie können in PostgreSQL sowohl einzelne Benutzer als auch Benutzergruppen erstellen. Der Vorteil von Gruppen ist, dass Sie diese mit bestimmten Rechten versehen können, welche dann automatisch allen Benutzern zugewiesen werden, die dieser Gruppe angehören.

Bei PostgreSQL werden die Benutzer und Gruppen unter dem Begriff „Rollen“ zusammengefasst. Sie werden allein dadurch unterschieden, dass eine Rolle, die als Benutzer dient, sich im Datenbanksystem anmelden kann, während andere Rollen das nicht können. Dabei können Rollen sowohl individuelle Zugriffsrechte besitzen als auch Teil einer anderen Rolle sein.

Neue Benutzer anlegen

Bei der Installation von PostgreSQL wird automatisch ein mit allen Rechten ausgestatteter Benutzer angelegt. In der Regel heißt dieser Benutzer *postgres*. Um weitere Benutzer zu erzeugen, müssen Sie sich zunächst mit diesem Benutzer anmelden. Anschließend können Sie mithilfe der Anweisung `CREATE ROLE` und der Option `LOGIN` einen Benutzer anlegen und, wenn gewünscht, einige Benutzerattribute festlegen.

Schulversion
© HERDT-Verlag

Beispiel: *BenutzerAnlegenPostgreSQL.sql*

Im folgenden Beispiel sollen die Benutzer *pscherhorn* und *jbossert* erstellt werden. Beide Benutzer sollen ein Passwort erhalten. Der Benutzer *pscherhorn* soll alle Rechte erhalten, also ein sogenannter Superuser sein. Der Benutzer *jbossert* darf Datenbanken erstellen. Zusätzlich sollen die Benutzer *fmueller* und *pfunke* erstellt werden, die zunächst keine Rechte haben.

```
① CREATE ROLE fmueller LOGIN; CREATE ROLE pfunke LOGIN;
② CREATE ROLE pscherhorn LOGIN PASSWORD 'passwort' SUPERUSER;
③ CREATE USER jbossert PASSWORD 'passwort' CREATEDB;
```

① Die Benutzer *fmueller* und *pfunke* werden ohne weitere Optionen angelegt.

② Der Benutzer *pscherhorn* wird angelegt. Mit der Anweisung `PASSWORD` geben Sie dem Benutzer ein Passwort mit. Die zusätzliche Anweisung `SUPERUSER` bewirkt, dass dieser Benutzer alle Zugriffsrechte erhält.

③ Hier wird der Benutzer *jbossert* erstellt. Auch diesem Benutzer wird ein Passwort zugewiesen und durch die Anweisung `CREATEDB` das Recht erteilt, Datenbanken zu erstellen. Da versäumt wurde, den Zusatz `LOGIN` einzugeben, wird sich der Benutzer nicht im Datenbanksystem anmelden können.

Möchten Sie in dem Namen Sonderzeichen oder Großbuchstaben verwenden, müssen Sie den Namen in Anführungszeichen setzen.

Syntax zum Anlegen neuer Benutzer

Die Anweisung `CREATE ROLE` ist sehr umfangreich. Hier ein Beispiel der wichtigsten Optionen:

```
CREATE ROLE benutzername LOGIN [[WITH]
      | [ENCRYPTED | UNENCRYPTED] PASSWORD 'passwort'
      | SUPERUSER | NOSUPERUSER
      | CREATEROLE | NOCREATEROLE
      | CREATEDB | NOCREATEDB
      | INHERIT | NOINHERIT
      | IN ROLE rollenname(n) [, ...]
    | VALID UNTIL 'datum' ];
```

- Neue Benutzer legen Sie mit der Anweisung `CREATE ROLE` gefolgt von einem eindeutigen Benutzernamen und der Option `LOGIN` an.

! Wenn Sie die Anweisung `LOGIN` nicht hinzufügen, erstellen Sie eine Rolle, die lediglich als Gruppe dient. Mit dieser Rolle können Sie sich nicht im Datenbanksystem anmelden.

Sie haben die Möglichkeit, weitere Attribute beim Anlegen des Benutzers festzulegen, z. B.:

`[ENCRYPTED|UNENCRYPTED] PASSWORD 'passwort'`	Der Benutzer erhält ein Passwort. Bei Angabe der Option `ENCRYPTED` wird das Passwort verschlüsselt. `UNENCRYPTED` ist der Vorgabewert.
`SUPERUSER | NOSUPERUSER`	Ist der Benutzer Superuser, erhält er alle Zugriffsrechte. `NOSUPERUSER` ist der Vorgabewert.

CREATEROLE | **NOCREATEROLE**	Mit der Option CREATEROLE erhält der Benutzer das Recht, Rollen zu erstellen.
CREATEDB | **NOCREATEDB**	Mit der Option CREATEDB erhält der Benutzer das Recht, Datenbanken zu erstellen.
INHERIT | **NOINHERIT**	Hier können Sie festlegen, ob der Benutzer automatisch alle Rechte der Rollen erben soll, denen er angehört.
IN ROLE *rollenname(n)*	Ordnet den Benutzer einer oder mehreren Rollen zu
VALID UNTIL *'datum'*	Beschränkt die Gültigkeit eines Passworts auf ein bestimmtes Datum
CONNECTION LIMIT *limitangabe*	Mit der Anweisung CONNECTION LIMIT können Sie ein Verbindungslimit eingeben, das die Anzahl der Datenbankverbindungen des Benutzers beschränkt.

Alternativ zu dem Befehl CREATE ROLE *name* LOGIN kann auch noch der Befehl CREATE USER verwendet werden. Diese Anweisung wurde in älteren Versionen von PostgreSQL verwendet.

Benutzerattribute anzeigen

Um sich einen Überblick über die Rollen im Datenbanksystem zu verschaffen, können Sie in PostgreSQL alle Rollen und deren Attribute mit dem Befehl "\du" anzeigen. In der Ansicht werden die Rollennamen aufgelistet sowie einige andere Informationen angegeben, z. B. ob die Rolle Superuser ist und ob sie Datenbanken erstellen darf.

```
postgres=# \du
                                   Liste der Rollen
 Rollenname |                         Attribute                         | Mitglied von
------------+-----------------------------------------------------------+--------------
 fmueller   |                                                           | {}
 jbossert   | DB erzeugen                                               | {}
 postgres   | Superuser, Rolle erzeugen, DB erzeugen, Replikation, Bypass RLS | {}
 pscherhorn | Superuser                                                 | {}
```

Rollen und deren Attribute anzeigen

Benutzerattribute ändern

Möchten Sie die Benutzer bzw. die Rollen bearbeiten, können Sie mit der Anweisung ALTER ROLE arbeiten. Bei dieser Anweisung können Sie u. a. die gleichen Optionen eingeben wie bei der Anweisung CREATE ROLE. Zusätzlich haben Sie z. B. die Möglichkeit, mit dem Befehl RENAME TO eine Rolle umzubenennen.

Beispiel: *BenutzerAendernPostgreSQL.sql*

```
① ALTER ROLE hschneider CREATEDB;
② ALTER USER jbossert LOGIN;
③ ALTER ROLE pscherhorn RENAME TO phscherhorn;
```

① Die Rolle *hschneider* kann nun Datenbanken erstellen.

② Die Rolle *jbossert* kann sich nun im Datenbanksystem einloggen.

③ Die Rolle *pscherhorn* wird umbenannt in *phscherhorn*.

© HERDT-Verlag

- Sie können nicht die Rolle ändern, mit der Sie aktuell eingeloggt sind.
- Wenn Sie eine Rolle umbenennen, die ein Passwort hat, wird dieses gelöscht.

Möchten Sie eine Rolle löschen, verwenden Sie die Anweisung `DROP ROLE rollenname`.

13.3 Benutzerverwaltung unter MariaDB

Neue Benutzer anlegen

Seit der Version 10.4.13 verwaltet MariaDB die Benutzernamen in der Tabelle mit dem Namen *global_priv* in der Datenbank *mysql*. Die bis zu dieser Version genutzte Tabelle *user* existiert als Sicht, welche die Tabelle *global_priv* referenziert, weiter. Um einen neuen Benutzer anzulegen, wird ein neuer Datensatz in dieser Tabelle erstellt. Um dies zu erreichen, bietet MariaDB verschiedene Wege an:

- die Verwendung der speziellen Befehle `CREATE USER` und `GRANT`, die den Server veranlassen, die Tabelle *user* zu bearbeiten;
- das direkte Einfügen der Datensätze mittels der SQL-Anweisung `INSERT` in die Tabelle *user*.

Die Verwendung der speziellen Befehle ist vorzuziehen, da diese klarer und weniger fehleranfällig sind.

Beim Anlegen des Benutzers müssen Sie neben Benutzernamen zusätzlich Angaben darüber machen, von welchen Computern (Hosts) im Netzwerk der Zugriff erfolgen darf. Die Vergabe eines Passwortes ist nicht zwingend erforderlich, sollte aber aus Gründen der Datensicherheit immer erfolgen. Bei der Verwendung von `GRANT` ist die Angabe eines Passwortes mittels der Option `IDENTIFIED BY` in Abhängigkeit von den Servereinstellungen unter Umständen jedoch notwendig.

Zum Einrichten eines neuen Benutzers benötigen Sie umfassende Rechte. Auf die Datenbank *mysql* haben nur Benutzer mit allen Rechten Zugriff, in der Regel daher nur der Datenbankadministrator.

Beispiel: *BenutzerAnlegenMariaDB.sql*

Im folgenden Beispiel sollen die Benutzer *fmueller* und *pfunke* erstellt werden. Der Benutzer *pfunke* darf nur von dem Computer aus auf die Datenbank zugreifen, auf dem sie installiert ist. Für den Benutzer *fmueller* ist dagegen Zugriff von beliebigen Computern des Netzwerks erlaubt.

```
① CREATE USER 'fmueller'@'%' IDENTIFIED BY 'geheim';
② CREATE USER 'pfunke'@'localhost' IDENTIFIED BY 'geheim';
```

① Hier wird der Benutzer *fmueller* erstellt. Dem Datenfeld *host* wird dabei der Platzhalter `%` zugewiesen. Dieser bewirkt, dass der Zugriff von beliebigen Computern im Netzwerk erfolgen kann. Das Passwort wird per Standard verschlüsselt in der Datenbank gespeichert. Dadurch ist das Passwort bei `SELECT`-Abfragen der Tabelle *user* auch für den Administrator nicht lesbar.

② Dem Benutzer *pfunke* sind nur Zugriffe vom lokalen Computer aus erlaubt. Deshalb wird dem Feld *host* der Wert *localhost* zugewiesen.

Syntax zum Anlegen neuer Benutzer

```
CREATE USER 'user'@'host' IDENTIFIED BY 'passwort' [PASSWORD
EXPIRE INTERVAL X DAY | PASSWORD EXPIRE NEVER];
```

- Neue Benutzer legen Sie mit der `CREATE USER`-Anweisung an.
- Den Benutzernamen und den Wert für das Datenfeld *host* geben Sie in Apostrophe eingeschlossen und getrennt durch das Zeichen '@' an. Der Hostname bezeichnet den Netzwerknamen des Computers, von dem aus der Zugriff erfolgen soll. Wird dieser nicht angegeben, wird '%' verwendet.
- Das Passwort für den Benutzer folgt ebenfalls in Apostrophe eingeschlossen nach den Schlüsselwörtern `IDENTIFIED BY`. Die Verschlüsselung des Passwortes in der Datenbank erfolgt automatisch.
- Seit der Version 10.4.3 kann über die Option `PASSWORD EXPIRE INTERVAL X DAY` bzw. `PASSWORD EXPIRE NEVER` die Dauer der Gültigkeit des Passworts bestimmt werden. Damit wird der für das per Systemvariabler gesetzte Wert für den zu definierende Benutzer überschrieben.

Beispiele für die Angabe des Hostnamens

Wert	Erklärung
`%`	Alle Computer im Netzwerk
`localhost`	Lokaler Computer, auf dem der MariaDB-Server installiert ist
`db.testserver.de`	Computer mit dem Domainnamen *db.testserver.de*
`%.testserver.de`	Computer, deren Domain auf *testserver.de* endet
`server`	Computer mit dem Netzwerknamen *server*

Die neuen Benutzer verfügen standardmäßig über keine Rechte. Diese können Sie nachträglich mit der Anweisung `GRANT` definieren.

Benutzer verwalten

Um Benutzerdaten unter MariaDB anzuzeigen, können Sie die Tabelle *mysql.user* mit einem `SELECT`-Befehl abfragen, beispielsweise in der Form

```
SELECT user, password FROM mysql.user;
```

Zum Löschen eines Benutzers nutzen Sie den Befehl

```
DROP USER 'user'
```

Zum Ändern vergebener Benutzerrechte empfiehlt es sich, die Befehle `GRANT` und `REVOKE` zu verwenden.

© HERDT-Verlag

Die direkte Bearbeitung der Tabelle *mysql.user* mit dem Befehl

```
UPDATE mysql.user SET datenfeld = wert, ...;
```

ist ebenfalls möglich.

13.4 Zugriffsrechte an Benutzer vergeben

Zugriffsrechte mit GRANT definieren

Standardmäßig kann außer dem Datenbankadministrator nur derjenige Benutzer ein Datenbankobjekt bearbeiten, der es auch erstellt hat. Um zusätzliche Zugriffsrechte zu definieren, wird die SQL-Anweisung GRANT verwendet.

Mit der Anweisung GRANT vergeben Sie Rechte für verschiedene SQL-Anweisungen, die sich auf ein ausgewähltes oder alle Datenbankobjekte beziehen. So können Sie beispielsweise einem Benutzer schreibenden Zugriff auf eine bestimmte Tabelle ermöglichen und einen lesenden Zugriff auf alle Tabellen und Datenbanken. Standardmäßig können Benutzerrechte nur vom Datenbankadministrator vergeben werden.

Beispiel: *Grant.sql*

Im folgenden Beispiel wird davon ausgegangen, dass die Benutzer *fmueller* und *pfunke* angelegt wurden. Dem Benutzer *fmueller* sollen alle Rechte für die Tabelle *t_ma* gewährt werden. Zusätzlich soll er Leserechte und Aktualisierungsrechte für die Tabelle *t_proj* erhalten. Der Benutzer *pfunke* soll für die Tabelle *t_ma* nur Leserechte erhalten.

```
① GRANT ALL ON t_ma TO fmueller;
  GRANT SELECT, UPDATE ON t_proj TO fmueller;
② GRANT SELECT ON t_ma TO 'pfunke'@'localhost';
```

① Hier werden mithilfe des Schlüsselworts ALL alle Rechte für die Tabelle *t_ma* an den Benutzer *fmueller* vergeben.

② Für den gleichen Benutzer wird für die Tabelle *t_proj* das Ausführen der SELECT- und UPDATE-Anweisung gestattet.

③ Der Benutzer *pfunke* erhält an dieser Stelle Leserechte für die Tabelle *t_ma*.

Syntax der GRANT-Anweisung

```
GRANT rechteliste ON datenbankobjekt TO benutzername;
```

- Die Anweisung beginnt mit dem Schlüsselwort GRANT.
- Danach folgt die Angabe des zu vergebenden Rechts. Sollen mehrere Rechte vergeben werden, müssen diese mit Kommata getrennt werden.
- Nach dem Schlüsselwort ON folgt der Name des Datenbankobjekts, für das die Rechte gelten sollen.
- Am Ende folgt nach dem Schlüsselwort TO der Benutzername.

© HERDT-Verlag Schulversion